

UNIVERSIDAD NACIONAL DEL COMAHUE

Centro Universitario Regional Zona Atlántica

Tecnicatura Superior en Administración de Sistemas y Software Libre

Trabajo Práctico 2

Scripts avanzados en Bash

Dato	Información
Asignatura	Automatización y Scripting
Alumno	Ulises Espinosa
Legajo	CURZA-4047
Fecha	27 de septiembre de 2026

Repositorio público: github.com/Nikcodra/curza-automatizacion-y-scripting

Índice

Índice	2
1. Introducción	3
1.1 Alcance y entregables	3
2. Enunciado completo	3
2.1 Descripción del caso práctico	3
2.2 Tarea 1: Buscador de versiones en PDF (pdf_version.sh)	3
2.3 Tarea 2: Renombrador y organizador automático (organizador.sh)	3
2.4 Tarea 3: Monitoreo interactivo de recursos (monitorear.sh)	3
2.5 Tarea 4: Verificador de sitios web (chequear_paginas.sh)	4
2.6 Tarea 5: Respalgador con prevención de ejecución simultánea (backup_manager.sh)	4
2.7 Entregables y observaciones	4
3. Hipótesis, supuestos y decisiones	4
4. Resolución	5
4.1 pdf_version.sh	5
4.2 organizador.sh	5
4.3 monitorear.sh	5
4.4 chequear_paginas.sh	5
4.5 backup_manager.sh	6
5. Pruebas y resultados	6
6. Repositorio e historial	7
7. Dificultades encontradas	7
8. Conclusión	7

1. Introducción

Este trabajo desarrolla cinco herramientas de administración en Bash para procesar archivos PDF, organizar archivos, consultar recursos del sistema, verificar sitios web y generar respaldos protegidos contra ejecuciones simultáneas. Los scripts fueron ejecutados y verificados en Ubuntu 24.04 sobre WSL 2.

La implementación utiliza condicionales, arreglos, bucles for, funciones, select, case, find, curl, trap, archivos de bloqueo y sustitución de cadenas. El código, las capturas reales, los registros y los respaldos se conservan en un repositorio público de GitHub.

1.1 Alcance y entregables

- Cinco scripts ejecutables dentro de código/.
- Archivo sitios_CURZA-4047.txt con cuatro URLs de prueba.
- Log chequeo_CURZA-4047.log con códigos 200, 302, 404 y 500.
- Respaldos comprimidos con los cinco scripts.
- README con instrucciones y capturas reales de terminal.
- Repositorio GitHub público con un historial de avances separados y mensajes claros.

2. Enunciado completo

2.1 Descripción del caso práctico

Como administrador de la red de laboratorios del CURZAS, se solicita automatizar la gestión y monitoreo diario de los servidores de archivos de la universidad. Se deben estructurar scripts que procesen extensiones de archivos de los alumnos, verifiquen el estado de servicios web locales, limpien directorios temporales de forma ordenada y prevengan colisiones al ejecutar tareas de respaldo redundantes.

2.2 Tarea 1: Buscador de versiones en PDF (pdf_version.sh)

- Buscar todos los archivos con extensión .pdf en el directorio actual y sus subdirectorios.
- Por cada archivo encontrado, extraer la primera línea mediante head -n 1.
- Imprimir: Archivo: [nombre.pdf] - Versión PDF: [1.X].
- Omitir cualquier PDF cuyo nombre contenga la palabra excluir o las iniciales del alumno.

2.3 Tarea 2: Renombrador y organizador automático (organizador.sh)

- Recibir un directorio destino como argumento, validar que exista y sea accesible y terminar con código 1 ante una entrada inválida.
- Crear imagenes/ para .jpg y .png; documentos/ para .pdf, .txt y .docx; comprimidos/ para .zip, .tar.gz y .rar; y otros/ para el resto.
- Mover cada archivo a la carpeta correspondiente utilizando bucles for y condicionales.
- Renombrar los archivos .old reemplazando esa terminación por .backup mediante sustitución de cadenas en Bash.

2.4 Tarea 3: Monitoreo interactivo de recursos (monitorear.sh)

- Implementar un panel interactivo utilizando select y case.
- Opción 1: mostrar la memoria RAM libre y usada en megabytes.
- Opción 2: buscar en HOME archivos mayores a 10 MB y mostrar los cinco más grandes.
- Opción 3: mostrar el uso formateado de las particiones físicas montadas.
- Opción 4: terminar con un saludo personalizado que incluya el legajo.

- Repetir el menú indefinidamente hasta seleccionar Salir.

2.5 Tarea 4: Verificador de sitios web (chequear_paginas.sh)

- Aceptar una lista de URLs como argumentos; si no se reciben, leer sitios_CURZA-4047.txt.
- Consultar silenciosamente cada URL con curl y obtener únicamente el código HTTP.
- Mostrar en verde el código 200, en amarillo los 3xx y en rojo los 4xx y 5xx.
- Guardar el reporte en logs/chequeo_CURZA-4047.log.

2.6 Tarea 5: Respalador con prevención de ejecución simultánea (backup_manager.sh)

- Crear atómicamente /var/lock/backup_CURZA-4047.lock mediante mkdir.
- Si el bloqueo ya existe, informar el error y terminar inmediatamente con código 9.
- Después de adquirir el bloqueo, registrar con trap su eliminación al finalizar.
- Buscar en el directorio de código los archivos modificados durante las últimas 24 horas.
- Copiarlos a /tmp/backup_CURZA-4047/ y generar logs/backup_CURZA-4047_FECHA.tar.gz.

2.7 Entregables y observaciones

- Informe PDF con carátula, identificación del alumno, índice, copia del enunciado, resolución, hipótesis, supuestos y aclaraciones.
- Código completo, correctamente indentado y comentado, dentro de un repositorio de GitHub.
- Carpeta tp2-espinosa-CURZA-4047/ con README.md, sitios_CURZA-4047.txt, codigo/ y logs/.
- README con capturas de ejecución y del menú interactivo.
- Historial de al menos tres commits con mensajes claros; se penaliza un único push final.
- Encabezado y pie en todas las páginas salvo la carátula; el pie debe indicar Página XX de YY.
- Interfaz de texto plano, sin código externo salvo indicación expresa y con validación de entradas para evitar fallos inesperados.

3. Hipótesis, supuestos y decisiones

- Las iniciales del alumno se implementan como ue, sin distinguir mayúsculas y minúsculas.
- El organizador procesa solamente los archivos regulares del primer nivel del directorio recibido para no volver a clasificar sus propias carpetas.
- Los nombres se obtienen con find -print0 y mapfile -d "", y luego se recorren con for para cumplir la consigna sin perder rutas con espacios.
- Un archivo .old se transforma en .backup y se mueve a otros/, ya que .backup no pertenece a las categorías indicadas.
- Ante nombres repetidos se agrega un sufijo numérico para impedir sobreescrituras.
- El verificador no sigue redirecciones, porque necesita conservar el estado 3xx para mostrarlo en amarillo.
- Los errores de conexión de curl se representan con 000 y se informan como Sin respuesta.
- El respaldo se ejecuta con autorización para crear el directorio de bloqueo en /var/lock.
- Las salidas controladas 1 y 9 se mantienen porque son exigidas expresamente por el enunciado.
- Aunque el texto menciona capturas de tres scripts, se documentaron los cinco componentes para cubrir todas las tareas.

4. Resolución

4.1 pdf_version.sh

El script recorre el directorio actual y sus subdirectorios con find. Convierte temporalmente cada nombre a minúsculas, aplica los filtros excluir y ue, lee la cabecera mediante head y extrae la versión con una expresión regular.

```
while IFS= read -r -d '' archivo; do
    nombre="${archivo##*/}"
    nombre_minusculas="${nombre,,}"
    [[ "$nombre_minusculas" == *excluir* || "$nombre_minusculas" == *ue* ]] && continue
    primera_linea=$(LC_ALL=C head -n 1 -- "$archivo" 2>/dev/null)
done < <(find . -type f -iname '*.pdf' -print0)
```

4.2 organizador.sh

Se valida un único argumento y los permisos del directorio. find produce una lista terminada en NUL; mapfile la carga en un arreglo y el bucle for procesa cada ruta. Un case clasifica los nombres y contempla la extensión compuesta .tar.gz. La función destino_libre evita sobrescribir archivos existentes.

```
mapfile -d '' -t archivos < <(find "$directorio" -maxdepth 1 -type f -print0)

for archivo in "${archivos[@]}"; do
    nombre="${archivo##*/}"
    nombre_minusculas="${nombre,,}"
    case "$nombre_minusculas" in
        *.jpg|*.png)      categoria="imagenes" ;;
        *.pdf|*.txt|*.docx) categoria="documentos" ;;
        *.zip|*.tar.gz|*.rar) categoria="comprimidos" ;;
        *)                categoria="otros" ;;
    esac
done
```

4.3 monitorear.sh

Las cuatro operaciones se separan en funciones. select genera el menú y case decide la acción. La búsqueda de archivos grandes utiliza find -size +10M, ordenamiento numérico y head -n 5. El menú solo termina al seleccionar Salir.

```
select opcion in "${opciones[@]}"; do
    case "$opcion" in
        "Monitorear memoria RAM") mostrar_memoria ;;
        "Buscar archivos grandes") buscar_archivos_grandes ;;
        "Espacio en particiones") mostrar_particiones ;;
        "Salir") echo "Hasta luego. Ulises Espinosa - CURZA-4047."; break ;;
        *) echo "Opción inválida: $REPLY." ;;
    esac
done
```

4.4 chequear_paginas.sh

El arreglo urls conserva cada argumento por separado o recibe las líneas válidas del archivo de sitios. curl descarta el cuerpo y devuelve el código HTTP. Los colores se aplican solamente a la terminal y el registro se escribe como texto plano.

```
codigo=$(curl -s -o /dev/null -w '%{http_code}' --connect-timeout 10 --max-time 20 "$url")

case "$codigo" in
    200) color="$VERDE"; estado="OK" ;;
    3??) color="$AMARILLO"; estado="REDIRECCIÓN" ;;
    4??|5??) color="$ROJO"; estado="ERROR HTTP" ;;
    *) color="$ROJO"; estado="SIN RESPUESTA" ;;
esac
```

4.5 backup_manager.sh

El bloqueo se adquiere antes de crear recursos temporales. El trap se registra únicamente después del mkdir exitoso, evitando que una segunda instancia elimine un bloqueo ajeno. Los archivos recientes se copian conservando su estructura relativa y luego se comprimen dentro de logs/.

```
if ! mkdir "/var/lock/backup_CURZA-4047.lock" 2>/dev/null; then
    echo "Error: ya existe una ejecución de backup_manager.sh." >&2
    exit 9
fi
trap 'rmdir "/var/lock/backup_CURZA-4047.lock"' EXIT

find . -mtime -1 -type f -print0
tar -czf "$archivo_backup" -C "$TEMP_DIR" .
```

5. Pruebas y resultados

Comprobación	Resultado	Estado
Sintaxis Bash	Los cinco scripts pasan bash -n	Correcto
Buscador PDF	Informa PDF válidos y omite excluir/UE	Correcto
Organizador	Bucle for y seis extensiones clasificadas	Correcto
Directorio inválido	organizador.sh devuelve código 1	Correcto
Monitor	RAM, archivos grandes, particiones y salida	Correcto
Estados HTTP	200, 302, 404 y 500 registrados	Correcto
Contenido del tar.gz	Incluye los cinco scripts	Correcto
Segunda instancia	Finaliza con código 9 y conserva el bloqueo ajeno	Correcto

La auditoría final se repitió en una copia temporal sobre Ubuntu 24.04. Los datos de prueba se eliminaron al finalizar y no alteraron los archivos versionados del repositorio.

6. Repositorio e historial

El código se encuentra en el repositorio público github.com/Nikcodra/curza-automatizacion-y-scripting. La carpeta de este trabajo es `tp2-espinosa-CURZA-4047/`. El historial supera los tres commits requeridos y conserva por separado la estructura inicial, los scripts, las pruebas, el informe, las capturas y las correcciones finales.

Etapa	Avance registrado
1	Preparar la estructura del trabajo práctico
2	Agregar el buscador PDF y el organizador
3	Incorporar el monitor y el chequeo web
4	Implementar el respaldo y su bloqueo
5	Documentar pruebas e incorporar el informe
6	Agregar capturas reales de las ejecuciones
7	Corregir el organizador para utilizar for

7. Dificultades encontradas

- La primera URL de prueba elegida no respondió desde WSL; se substituyó por `httpbin.org/status/200` para conservar un caso verificable.
- La extensión `.tar.gz` exige comparar el nombre completo, porque la última extensión aislada sería solamente `.gz`.
- El trap debe registrarse después de adquirir el bloqueo para que una instancia rechazada no elimine el directorio de otra ejecución.
- La exigencia de usar `for` se resolvió cargando rutas NUL en un arreglo, evitando romper nombres que contienen espacios.

8. Conclusión

Los cinco scripts cumplen las tareas solicitadas y fueron probados en Ubuntu. La solución valida las entradas, conserva nombres con espacios, clasifica extensiones simples y compuestas, mantiene un registro HTTP legible y evita respaldos simultáneos mediante una operación atómica. El repositorio público conserva la documentación, las evidencias y un historial de avances funcionales.